

#04 Data Structures I

김성수

4thrunyul@gmail.com

0. 목차



1. Linear Data Structures

- Stack
- Queue
- Deque

2. Associative Data Structures

- Beyond Searching
- Set
- Map

3. Priority Data Structures

- Priority Queue
- Heap

1. Linear Data Structures

1. Linear Data Structures



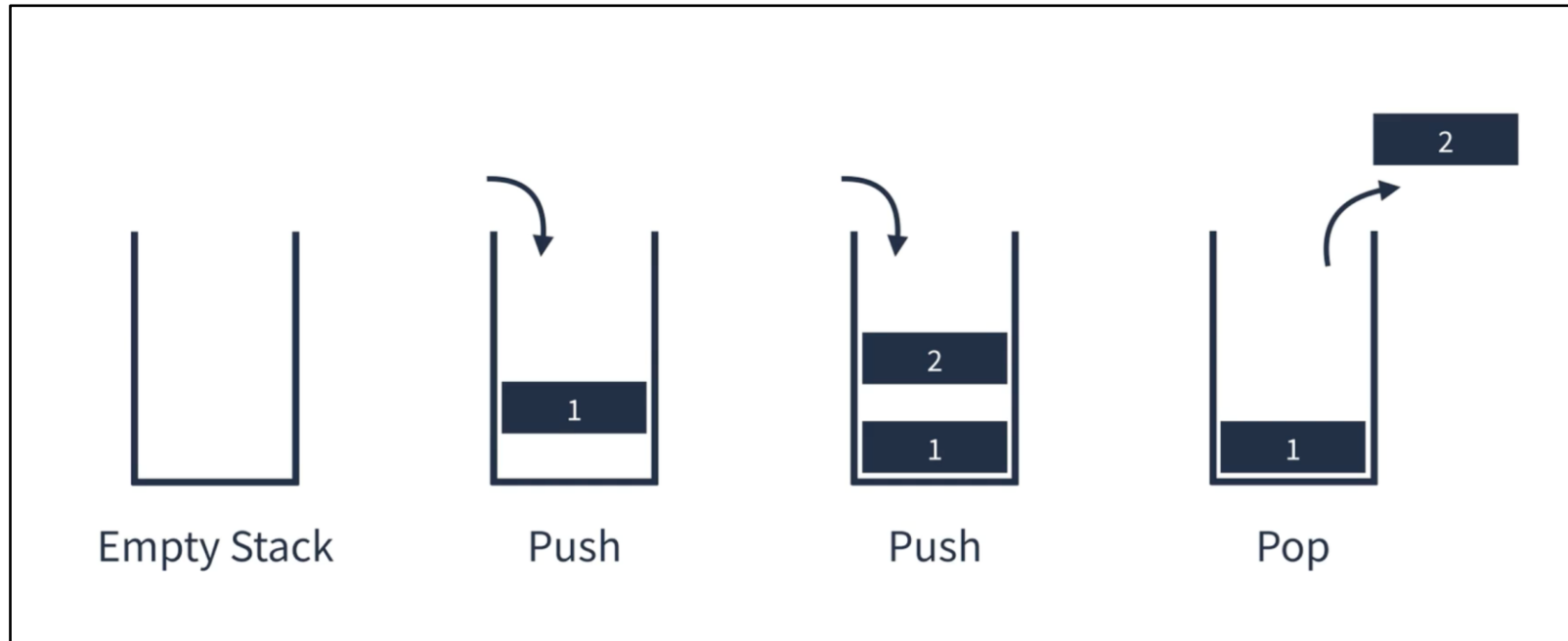
Linear Data Structures(선형 자료구조)

- 데이터를 일렬로 순차적으로 나열하여 요소 간의 관계가 1:1로 연결된 자료구조
- 데이터가 메모리 상에 물리적 또는 논리적으로 연속되어 저장되므로 순서대로 데이터를 흐르게 제어함
- stack, queue, deque이 이에 해당 됨

1. Stack

Stack(스택)

- 데이터를 한쪽 끝에서만 넣고 뺄 수 있는 후입선출(LIFO, Last-In First-Out) 방식의 자료구조
- 컴퓨터는 stack을 통해 연산 순서를 정렬함(중위 표기식 -> 후위 표기식)



1. Stack

```

1  #include <iostream>
2  #include <stack>
3
4  using namespace std;
5
6  int main() {
7      stack<int> stk;
8
9      stk.push(1);
10     stk.push(2);
11     stk.pop();
12
13     return 0;
14 }

```

stack<자료형> 변수명 | <stack>

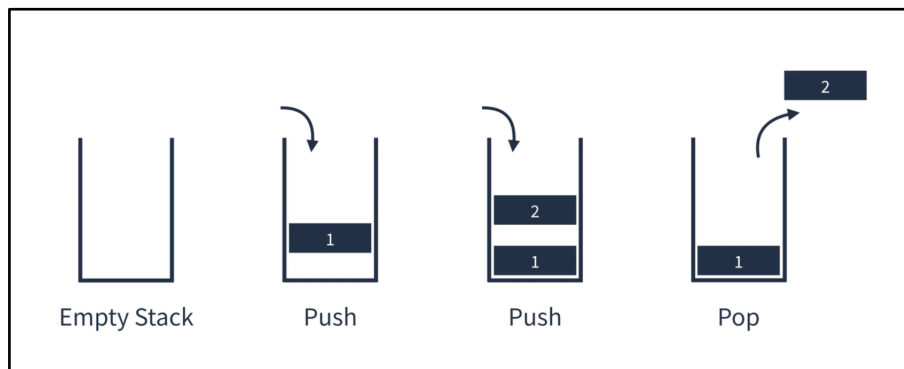
- 스택을 선언하기 위해서는 <stack> 라이브러리 필요
- 자료형에는 거의 모든 자료형을 사용할 수도 있으며, 구조체를 선언하여 자료형으로 넣을 수도 있음
- stack<자료형> 변수명({요소1, 요소2, 요소3}) 처럼 초기 선언도 가능하며, 이 경우 요소1부터 순서대로 push함.

stk.push(value) | <stack> | O(1)

- 스택의 가장 윗부분에 새로운 데이터를 추가하여 저장하는 함수

stk.pop() | <stack> | O(1)

- 스택의 가장 윗부분에 위치한 데이터를 제거하는 작업
- 비어 있는 스택에 pop을 사용하면 프로그램이 정지되며 아무것도 출력되지 않음



1. Stack

```
1  #include <iostream>
2  #include <stack>
3
4  using namespace std;
5
6  int main() {
7      stack<int> stk;
8
9      stk.push(1);
10     stk.push(2);
11
12     cout << stk.top() << "\n";
13     cout << stk.size() << "\n";
14     cout << stk.empty();
15
16     return 0;
17 }
```

Input:

Copy

Expected Output:

Copy

2
2
0

`stk.top()` | `<stack>` | $O(1)$

- 스택의 가장 윗부분에 위치한 데이터를 조회하는 함수
- 비어 있는 스택에 top을 사용하면 프로그램이 정지되며 아무것도 출력되지 않음

`stk.size()` | `<stack>` | $O(1)$

- 스택에 현재 저장되어 있는 데이터의 총 개수를 확인하는 함수
- 내부 컨테이너에 요소의 수를 저장하고 있어서 $O(1)$ 의 시간 복잡도를 가짐

`stk.empty()` | `<stack>` | $O(1)$

- 스택에 저장된 데이터가 없는지 여부를 확인하는 함수
- bool형으로 반환하며, 스택이 비어 있으면 true, 요소가 하나도 존재한다면 false를 반환

스택의 초기화 | $O(n)$

- 스택에는 `stk.clear()` 함수가 존재하지 않음
- 스택을 초기화하기 위해서는 재선언 하거나 `stk = {}`으로 초기화할 수 있음
- 단, `stk = {1, 2, 3}` 같은 방법은 통하지 않고, `stk = {}`만 예외적으로 가능함.

1. Stack: 실습(BOJ)

- [28278 스택 2](#)
- [코드](#)

```
1  #include <iostream>
2  #include <stack>
3
4  using namespace std;
5
6  int main() {
7      ios_base::sync_with_stdio(false);
8      cin.tie(NULL);
9
10     int n, com, num;
11     stack<int> stk;
12
13     cin >> n;
```

```
15     for(int i = 0; i < n; i++) {
16         cin >> com;
17
18         if(com == 1) {
19             cin >> num;
20
21             stk.push(num);
22         }
23         else if(com == 3) {
24             cout << stk.size() << "\n";
25         }
26         else if(!stk.empty()) {
27             if(com == 2) {
28                 cout << stk.top() << "\n";
29
30                 stk.pop();
31             }
32             else if(com == 5) cout << stk.top() << "\n";
33             else if(com == 4) cout << stk.empty() << "\n";
34         }
35         else {
36             if(com == 4) cout << stk.empty() << "\n";
37             else cout << "-1\n";
38         }
39     }
40
41     return 0;
42 }
```


1. Stack: 실습(BOJ)



실습

- [10773 제로](#)
 - [코드](#)
- [9012 괄호](#)
 - [코드](#)
 - Stack을 사용하지 않고 푸는 방법도 고민해보기

1. Stack: 실습(정올)

- 1102 스택
 - 코드

```
1  #include <iostream>
2  #include <stack>
3
4  using namespace std;
5
6  int main() {
7      // 변수 선언
8      int n, a;
9      char c;
10     stack<int> stk;
11
12     cin >> n;
13
14     while(n-- > 0) {
15         // 명령 입력
16         cin >> c;
```

```
18         // 명령이 'a'일 때만 입력이 추가로 필요
19         if(c == 'a') {
20             cin >> a;
21
22             stk.push(a);
23         }
24         else if(c == 'c') cout << stk.size() << "\n";
25         else if(c == 'o') {
26             // 비어있는 stack에 .top(), .pop() 사용시 프로그램이 정지되니 주의
27             // 즉, .empty()로 확인 필요
28             if(stk.empty()) cout << "empty\n";
29             else {
30                 cout << stk.top() << "\n";
31
32                 stk.pop();
33             }
34         }
35     }
36
37     return 0;
38 }
```

1. Stack: 실습(정올)



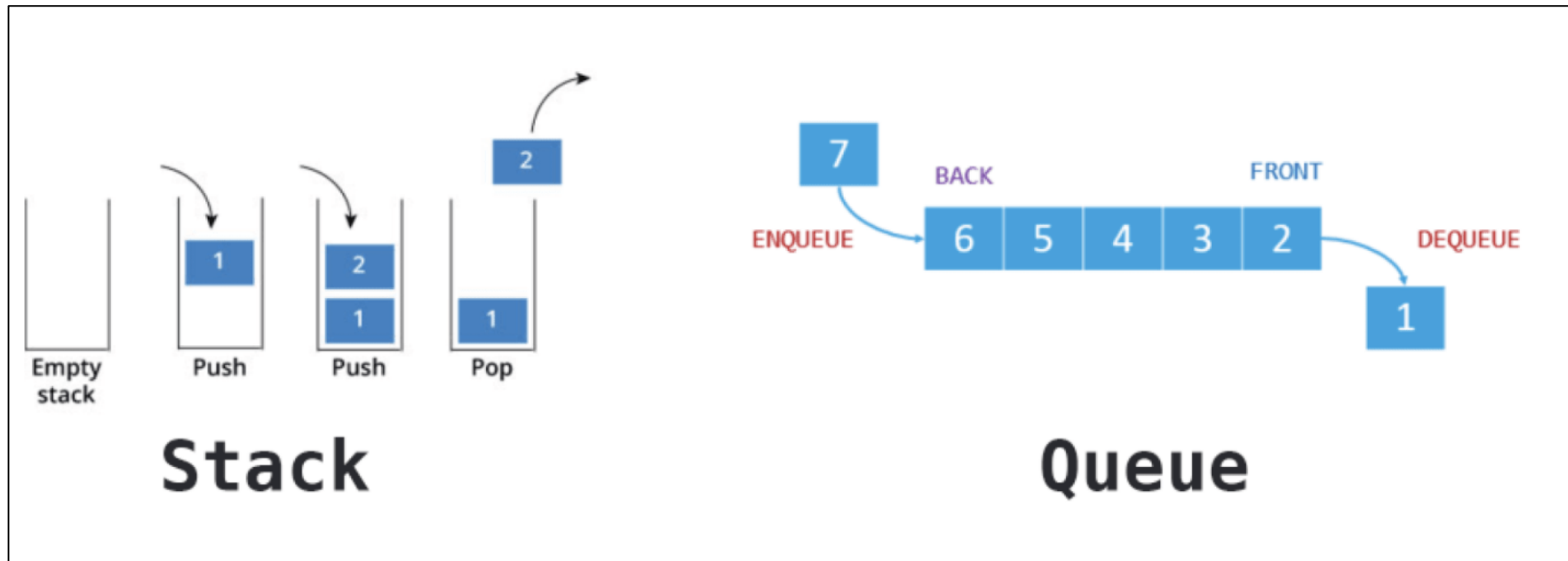
실습

- [2858 쇠막대기](#)
 - 코드
- [5917 접시 닦기](#)
 - 코드
- [1221 후위표기법1](#)
 - 코드

1. Queue

Queue(큐)

- 데이터를 한쪽 끝(뒤)에서 넣고 반대쪽 끝(앞)에서 뺄 수 있는 선입선출(FIFO, First-In First-Out) 방식의 자료구조



1. Queue

```

1  #include <iostream>
2  #include <queue>
3
4  using namespace std;
5
6  int main() {
7      queue<int> q({1, 2, 3, 4, 5, 6});
8
9      q.push(7);
10     q.pop();
11
12     return 0;
13 }

```

queue<자료형> 변수명 | <queue>

- 큐를 선언하기 위해서는 <queue> 라이브러리 필요
- 자료형에는 거의 모든 자료형을 사용할 수도 있으며, 구조체를 선언하여 자료형으로 넣을 수도 있음
- queue<자료형> 변수명({요소1, 요소2, 요소3}) 처럼 초기 선언도 가능하며, 이 경우 요소1부터 순서대로 push함.

q.push(value) | <queue> | O(1)

- 큐의 뒷부분(Rear)에 새로운 요소를 추가하여 저장하는 작업
- 이론적으로는 enqueue라고 하지만, C++에서는 통일성을 위해 push로 통일 됨

q.pop() | <queue> | O(1)

- 큐의 가장 앞부분(Front)에 위치한 요소를 제거하는 작업
- 비어 있는 큐에 pop을 사용하면 프로그램이 정지되며 아무것도 출력되지 않음
- 이론적으로는 dequeue라고 하지만, C++에서는 통일성을 위해 pop으로 통일 됨



1. Queue

```
1  #include <iostream>
2  #include <queue>
3
4  using namespace std;
5
6  int main() {
7      queue<int> q({5, 8, 4, 3, 2});
8
9      cout << q.front() << "\n";
10     cout << q.back() << "\n";
11     cout << q.size() << "\n";
12
13     q.push(q.front());
14
15     cout << q.back() << "\n";
16     cout << q.empty();
17 }
```

Input: Copy

Expected Output: Copy

```
5
2
5
5
0
```

q.front() | <queue> | O(1)

- 큐의 가장 앞에 위치한 데이터를 조회하는 함수
- 비어 있는 큐에 front를 사용하면 프로그램이 정지되며 아무것도 출력되지 않음

q.back() | <queue> | O(1)

- 큐의 가장 뒷부분에 위치한 데이터를 조회하는 함수
- 비어 있는 큐에 back을 사용하면 프로그램이 정지되며 아무것도 출력되지 않음

q.size() | <queue> | O(1)

- 큐에 현재 저장되어 있는 데이터의 총 개수를 확인하는 함수
- 내부 컨테이너에 요소의 수를 저장하고 있어서 O(1)의 시간 복잡도를 가짐

q.empty() | <queue> | O(1)

- 큐에 저장된 데이터가 없는지 여부를 확인하는 함수
- bool형으로 반환하며, 큐가 비어 있으면 true, 요소가 하나라도 존재한다면 false를 반환

큐의 초기화 | O(n)

- 큐에는 q.clear() 함수가 존재하지 않음
- 큐를 초기화하기 위해서는 재선언 하거나 q = {}으로 초기화할 수 있음
- 단, q = {1, 2, 3} 같은 방법은 통하지 않고, q = {}만 예외적으로 가능함.

1. Queue

- 18258 큐 2
- 코드

```
1  #include <iostream>
2  #include <queue>
3  #include <string>
4
5  using namespace std;
6
7  int main() {
8      ios_base::sync_with_stdio(false);
9      cin.tie(NULL);
10
11     int n, num;
12     string s;
13     queue<int> q;
14
15     cin >> n;
```

```
17     for(int i = 0; i < n; i++) {
18         cin >> s;
19
20         if(s == "push") {
21             cin >> num;
22
23             q.push(num);
24         }
25         else if(s == "size") cout << q.size() << "\n";
26         else if(s == "empty") {
27             if(q.empty()) cout << "1\n";
28             else cout << "0\n";
29         }
30         else if(!q.empty()) {
31             if(s == "pop") {
32                 cout << q.front() << "\n";
33
34                 q.pop();
35             }
36             else if(s == "front") cout << q.front() << "\n";
37             else cout << q.back() << "\n";
38         }
39         else cout << "-1\n";
40     }
41 }
```

1. Queue: 실습(BOJ)



실습

- [2164 카드2](#)
 - [코드](#)
- [15828 Router](#)
 - [코드](#)

1. Queue: 실습(정올)

- 1697 큐
- 코드

```
1  #include <iostream>
2  #include <queue>
3
4  using namespace std;
5
6  int main() {
7      // 변수 선언
8      int n, num;
9      char c;
10     queue<int> q;
11
12     cin >> n;
13
14     for(int i = 0; i < n; ++i) {
15         // 명령 입력
16         cin >> c;
```

```
18         // 명령이 'i'일 때만 입력이 추가로 필요
19         if(c == 'i') {
20             cin >> num;
21
22             q.push(num);
23         }
24         else if(c == 'o') {
25             // 비어있는 queue에 .front(), .back(), .pop() 사용시 프로그램이 정지되니 주의
26             // 즉, .empty()로 확인 필요
27             if(q.empty()) cout << "empty\n";
28             else {
29                 cout << q.front() << "\n";
30
31                 q.pop();
32             }
33         }
34         else cout << q.size() << "\n";
35     }
36
37     return 0;
38 }
```

1. Queue: 실습(정올)



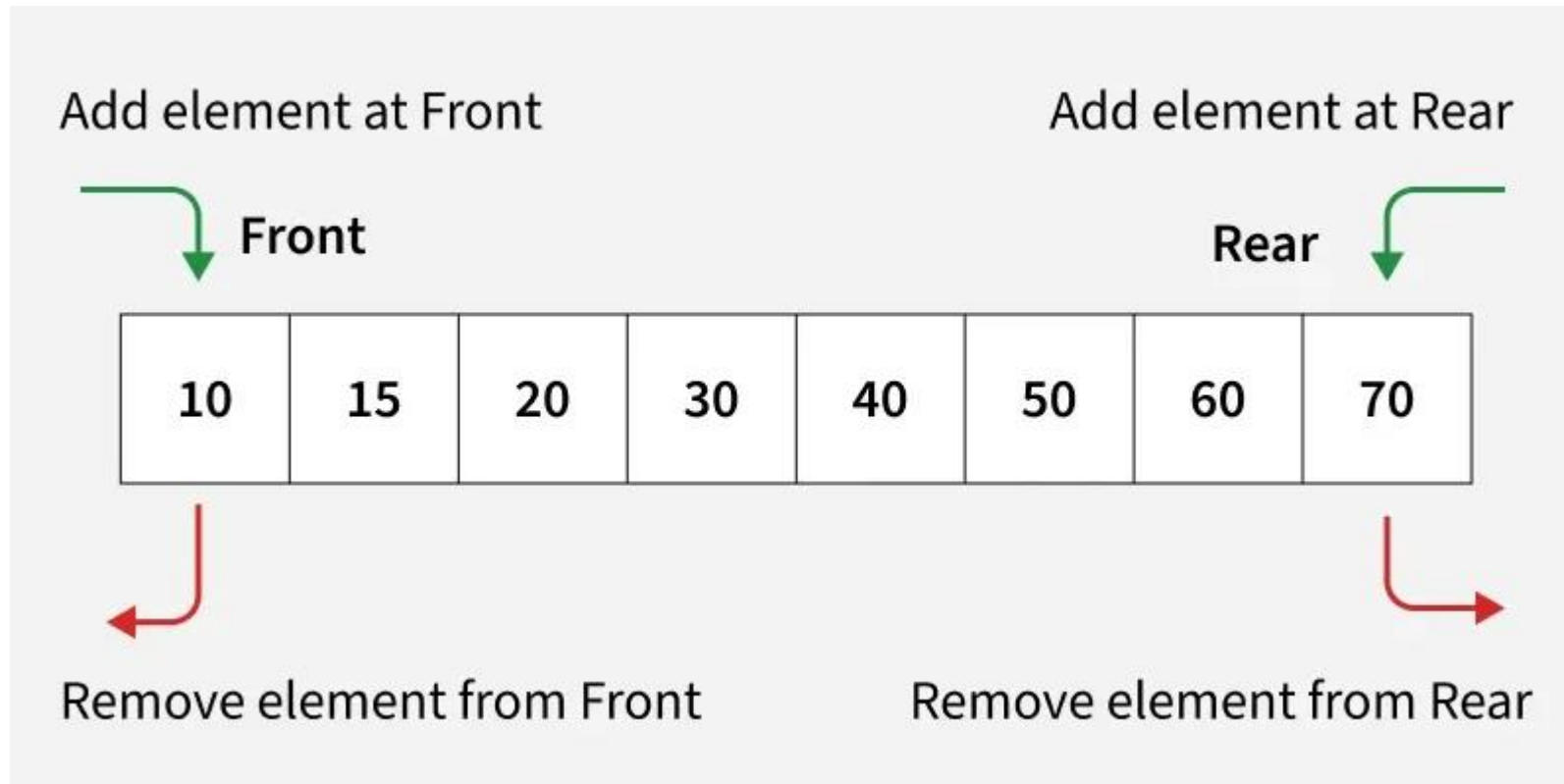
실습

- [5801 단순한 카드 셔플](#)
 - 코드
- [6057 피자왕 비용](#)
 - 코드
- [5527 콜센터](#)
 - 코드

1. Deque

Deque(덱/데크)

- 양쪽 끝에서 데이터를 넣고 뺄 수 있는 양방향 자료구조 (Double-Ended Queue)



1. Deque

deque<자료형> 변수명 | <deque>

- 덱을 선언하기 위해서는 <deque> 라이브러리 필요
- 자료형에는 거의 모든 자료형을 사용할 수도 있으며, 구조체를 선언하여 자료형으로 넣을 수도 있음

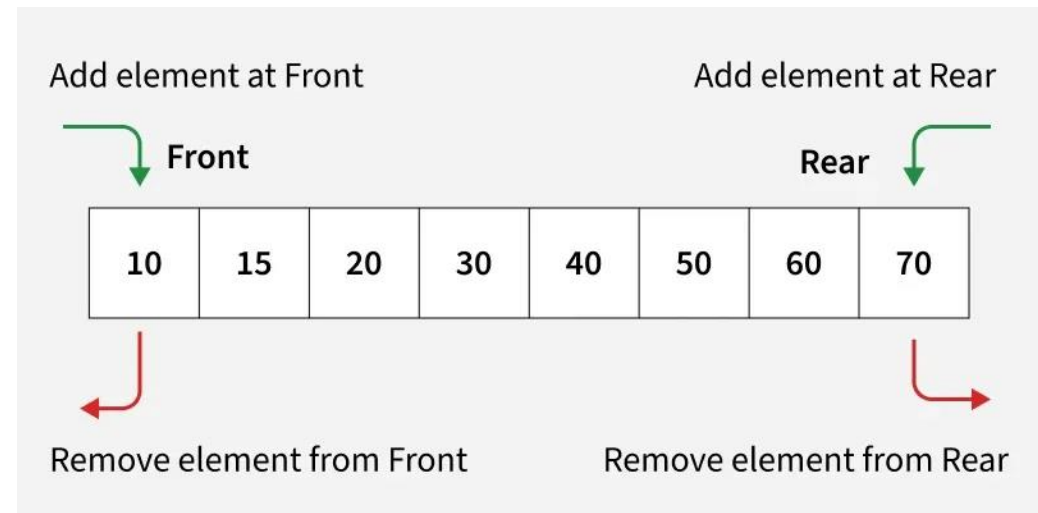
deq.push_front(요소), deq.push_back(요소) | <deque> | $O(1)$

- 덱의 가장 앞, 뒷부분에 새로운 데이터를 추가하여 저장하는 함수

deq.pop_front(), deq.pop_back() | <deque> | $O(1)$

- 덱의 가장 앞, 뒷부분에 위치한 데이터를 제거하는 작업
- 비어 있는 덱에 pop을 사용하면 프로그램이 정지되며 아무것도 출력되지 않음

```
1  #include <iostream>
2  #include <deque>
3
4  using namespace std;
5
6  int main() {
7      deque<int> deq({15, 20, 30, 40, 50, 60});
8
9      deq.push_front(10);
10     deq.push_back(70);
11
12     deq.pop_front();
13     deq.pop_back();
14
15     return 0;
16 }
```



1. Deque

```
1  #include <iostream>
2  #include <deque>
3
4  using namespace std;
5
6  int main() {
7      deque<char> deq({'h', 'e', 'l', 'l', 'o'});
8
9      cout << deq.front() << "\n";
10     cout << deq.back() << "\n";
11     cout << deq.size() << "\n";
12     cout << deq.empty();
13
14     return 0;
15 }
```

Input:

Copy

Expected Output:

Copy

h
o
5
0

deq.front(), deq.back() | `<deque>` | $O(1)$

- 덱의 가장 윗부분에 위치한 데이터를 조회하는 함수
- 비어 있는 덱에 조회 함수를 사용하면 프로그램이 정지되며 아무것도 출력되지 않음

deq.size() | `<deque>` | $O(1)$

- 덱에 현재 저장되어 있는 데이터의 총 개수를 확인하는 함수
- 내부 컨테이너에 요소의 수를 저장하고 있어서 $O(1)$ 의 시간 복잡도를 가짐

deq.empty() | `<deque>` | $O(1)$

- 덱에 저장된 데이터가 없는지 여부를 확인하는 함수
- bool형으로 반환하며, stack이 비어 있으면 true, 요소가 하나도 존재한다면 false를 반환

1. Deque: 차이점을 중심으로

```
1  #include <iostream>
2  #include <deque>
3
4  using namespace std;
5
6  int main() {
7      deque<int> deq = {3, 2, 1};
8
9      cout << deq.front() << "\n";
10
11     deq = {4, 5, 6};
12
13     cout << deq.front() << "\n";
14     cout << deq[1] << "\n";
15
16     deq.clear();
17
18     cout << deq.empty();
19
20     return 0;
21 }
```

deq.clear() | <deque> | $O(N)$

- deq.clear()를 통해 덱의 내부 요소를 제거하는 초기화 작업을 할 수 있음

덱의 초기값 | <deque> | $O(1)$

- 스택과 큐에서는 생성자를 통해서만 초기값을 지정할 수 있었던 것과 다르게 자유롭게 초기값을 재 지정할 수 있음

덱의 인덱싱 | <deque> | $O(1)$

- 스택과 큐는 인덱싱이 불가능 했던 것과 다르게 벡터처럼 인덱싱이 가능함.

Input:	Copy
Expected Output:	Copy
3	
4	
5	
1	

1. Deque: 차이점을 중심으로

Deque이 Vector를 대체할 수 있을까?

- vector의 `.insert()`나 `.erase()`는 $O(n)$ 의 시간 복잡도를 가지고 있음.
- 그에 반해 deque의 `.push_front()`, `.pop_front()`는 $O(1)$ 의 시간 복잡도를 가지고 있음.
- deque이 vector에 비해 성능이 같거나 어떤 부분에서는 더 좋은 게 아닌가?
- 메모리 저장 방식
 - vector
 - 메모리 상에 처음부터 끝까지 빈틈없이 하나의 연속된 덩어리로 데이터를 저장
 - deque
 - 일정한 크기의 작은 메모리 블록(Chunk)들을 여러 개 쪼개서 데이터를 나누어 저장
 - 이 블록들의 주소를 관리하는 중앙 컨트롤러(Map/Pointer Array)를 따로 둠
 - 앞이나 뒤에 데이터를 추가하면 새로운 작은 메모리 블록만 하나 더 할당
 - ex : 4byte 정수 6개
 - vector : 1000~1023(연속된 공간)
 - deque
 - 7000~7011 (Map/Pointer array)
 - 1000~1007(chunk 1), 3000~3007(chunk 2), 5000~5007(chunk 3)

1. Deque: 차이점을 중심으로

Deque이 Vector를 대체할 수 있을까?

- 포인터 간접 참조
 - vector
 - 원소에 접근할 때 시작 주소 + 인덱스 계산 한 번으로 즉시 원소에 도달
 - deque
 - 똑같은 $O(1)$ 접근이더라도 내부적으로 중앙 컨트롤러에서 해당 블록 주소를 찾고, 그 블록 안에서 인덱스를 찾는 두 단계를 거쳐야 함
- 캐시 효율성 및 캐시 적중률(Cache Hit Rate)
 - vector
 - 모든 데이터가 물리적으로 완전히 붙어 있어 연속된 데이터를 한꺼번에 캐시 메모리에 올리는 CPU 캐시 적중률이 극대화
 - deque
 - 블록 내부끼리는 붙어 있지만 블록과 블록 사이는 메모리 공간 여기저기에 파편화되어 흩어져 있기 때문에 벡터에 비해 캐시 효율이 떨어짐

1. Deque: 사실은...

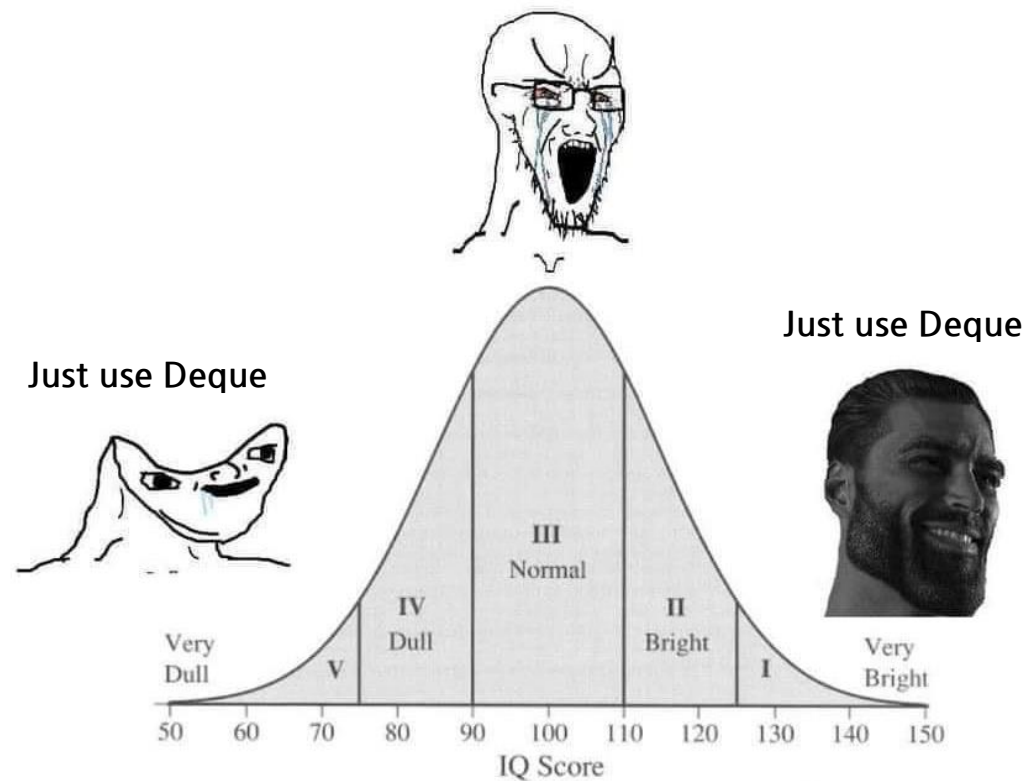
C++에서 스택과 큐는 내부적으로 덱을 사용함

- 덱에서 각 자료구조에 맞게 기능을 제한한 형태임.
- 시간복잡도, 공간복잡도 관점에서 차이가 전혀 없음.

이 문제는 후입선출을 요구하는 문제니 스택을 써야하고,
저 문제는 선입선출을 요구하니 큐를 쓰고...

그럼에도 스택과 큐를 사용하는 이유

- 꼭 필요한 기능만 남겨두어 실수를 차단하는 것에 목적이 있음



1. Deque: 실습(BOJ)

- [28279](#) 덱 2
- [코드](#)

```
1  #include <iostream>
2  #include <deque>
3
4  using namespace std;
5
6  int main() {
7      ios::sync_with_stdio(false);
8      cin.tie(NULL);
9
10     int n, com, num;
11     deque<int> deq;
12
13     cin >> n;
```

```
15     for(int i = 0; i < n; i++) {
16         cin >> com;
17
18         if(com == 1) {
19             cin >> num;
20
21             deq.push_front(num);
22         }
23         else if(com == 2) {
24             cin >> num;
25
26             deq.push_back(num);
27         }
28         else if(com == 5) cout << deq.size() << "\n";
29         else if(com == 6) {
30             if(deq.empty()) cout << "-1 \n";
31             else cout << "0 \n";
32         }
```

```
33     else {
34         if(deq.empty()) cout << "-1 \n";
35         else {
36             if(com == 3) {
37                 cout << deq.front() << "\n";
38
39                 deq.pop_front();
40             }
41             else if(com == 4) {
42                 cout << deq.back() << "\n";
43
44                 deq.pop_back();
45             }
46             else if(com == 7) cout << deq.front() << "\n";
47             else cout << deq.back() << "\n";
48         }
49     }
50 }
```

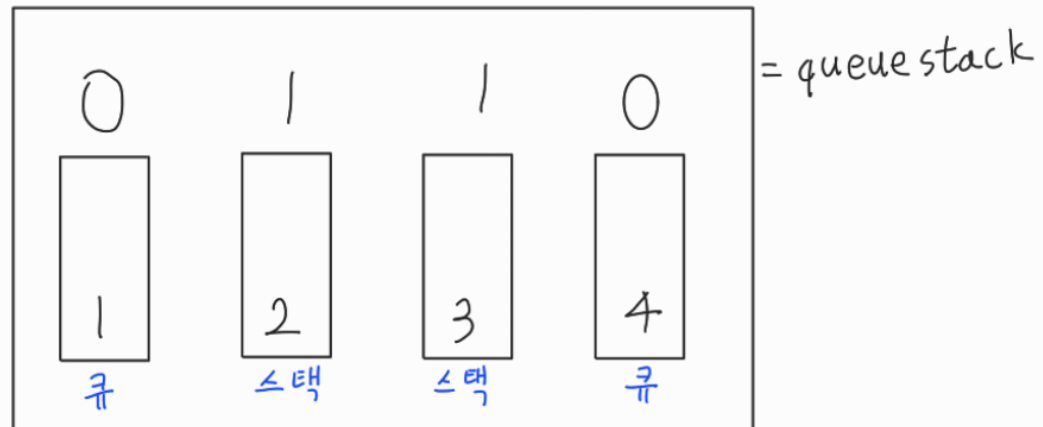
1. Deque: 실습(BOJ)

실습

- [24511 queuestack](#)
- [코드](#)

```
4
0 1 1 0
1 2 3 4
```

각 자료구조(0 : queue, 1 : stack)에 값을 하나씩 넣어두는 과정이라고 생각하면 된다.
이렇게 개별적인 큐와 스택을 모아둔 전체를 이 문제에서는 queuestack이라고 한다.



2. Associative Data Structures

2. Associative Data Structures



Associative Data Structures(연관 자료구조)

- 데이터 자체의 입력 순서가 아니라, 키(Key)와 값(Value) 또는 속성 간의 관계를 기준으로 데이터를 관리하는 자료구조
- 내부적으로 트리나 해싱 구조를 활용하여 순서와 상관없이 특정 키나 유일한 값을 찾아내며 탐색 속도를 극대화함
- set, map이 이에 해당 됨

2. Beyond Searching

Red-Black Tree

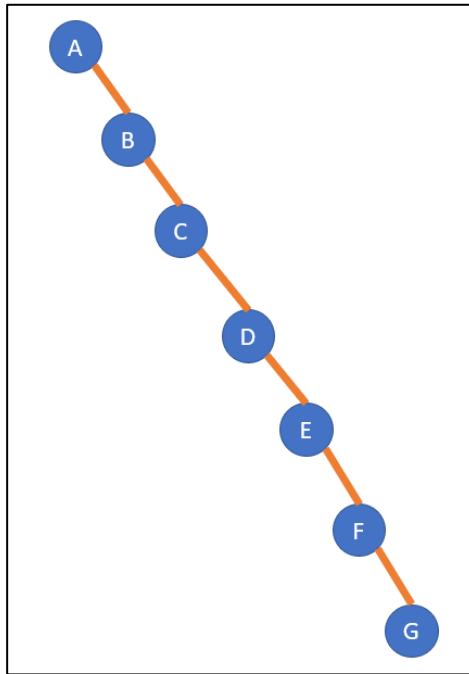
- 정렬된 데이터 셋에서 최선의 탐색 방법 → 이분 탐색, $O(\log n)$
- 탐색은 그렇다 치고, 삽입, 수정은?
- 기존 vector, array에서는 삽입, 수정을 하기 위해서는 $O(n)$
 - 아래 예제에서 1을 삽입하되, 정렬을 유지하기 위해서는 $O(n)$ 만큼 시간이 걸림

$$v = \{7, 64, 124, 156, 1055, 7594\}$$

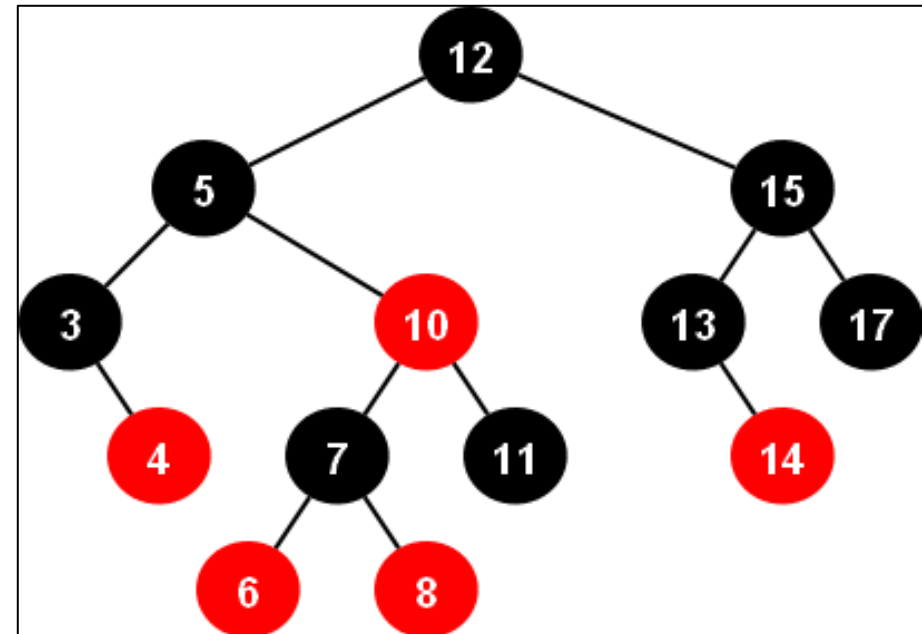
2. Beyond Searching

Red-Black Tree

- 자가 균형 이진 탐색 트리
- 탐색, 삽입, 수정 $O(\log n)$



▲ 편향된 이진 탐색 트리



▲ RB Tree

2. Beyond Searching



Hashing(해싱)

- 선형 탐색 : $O(n)$
- 이분 탐색
 - 정렬이 되어있는 경우 : $O(\log n)$
 - 정렬이 안 되어있는 경우 : $O(n \log n)$
- 더 빠르게 탐색하는 방법이 있을까?

$$v = \{54, 7, 59, 2952, 23, 0, 6\}$$

2. Beyond Searching

Hashing(해싱)

- 데이터를 해시 함수를 이용해 고정된 값(해시값)으로 변환하고, 이를 통해 데이터를 빠르게 저장하거나 탐색하는 기법
- 연산 한 번을 통해 해당 값 존재 여부를 확인할 수 있음 => 평균 $O(1)$
- 아래 예제에서 해시 함수는 $f(n) = n \% 10$ 이라고 설정

$v = \{54, 7, 59, 2952, 23, 0, 6\}$

hash table = $\{4, 7, 9, 2, 3, 0, 6\}$

2. Beyond Searching

Hashing(해싱)

- 해시 충돌(Hash Collision) 가능성을 배제할 수 없음
- 충돌을 막기 위해 보통 해시 함수에서 나머지 연산은 소수(Prime Number)를 사용
- 충돌이 일어나면 Chainig, Open Addressing 방법을 통해 제어 => 최악 $O(n)$
- 아래 예제에서 해시 함수는 $f(n) = n \% 10$ 이라고 설정
- 여기서 33을 해싱하면 기존 23의 해시값과 동일하여 충돌이 발생

$v = \{54, 7, 59, 2952, 23, 0, 6\}$

hash table = $\{4, 7, 9, 2, 3, 0, 6\}$

2. Beyond Searching



RB Tree vs Hash Table

- 오늘 배울 set과 map은 기본적으로 RB Tree 형태로 이루어져 있음
- 하지만 unordered_set, unordered_map은 Hash Table 형태로 이루어져 있음
- RB Tree
 - 데이터의 순서가 중요한 경우
 - 일관된 성능이 필요할 때(모든 연산이 $O(\log N)$ 이 보장됨)
- Hash Table
 - 데이터의 양이 방대하고, 단순 탐색 성능이 가장 중요한 경우
 - 메모리 공간보다 시간이 중요할 때

2. Set



Set(셋, 세트)

- 데이터를 중복 없이 저장하는 자료구조
- 내부적으로는 레드 블랙 트리(Red-Black Tree)로 구성되어 삽입과 동시에 자동으로 오름차순 정렬됨.

2. Set

```

1  #include <iostream>
2  #include <set>
3
4  using namespace std;
5
6  int main() {
7      set<int> s = {5, 2, 3, 2, 6};
8
9      s.insert(7); // {2, 3, 5, 6, 7}
10     s.erase(2); // {3, 5, 6, 7}
11
12     cout << s.erase(6) << "\n"; // {3, 5, 7}
13     cout << s.erase(2) << "\n"; // {3, 5, 7}
14     cout << s.size();
15
16     return 0;
17 }

```

Input:

Copy

Expected Output:

Copy

1
0
3

set<자료형> 변수명 | <set>

- 셋을 선언하기 위해서는 <set> 라이브러리 필요
- 자료형에는 거의 모든 자료형을 사용할 수도 있으며, 구조체를 선언하여 자료형으로 넣을 수도 있음

set.insert(val) | <set> | $O(\log n)$

- 셋에 새로운 데이터를 추가하여 저장하며, 자동으로 오름차순 정렬을 유지함
- 셋에 이미 존재하는 원소를 삽입할 경우 무시됨

set.erase(val) | <set> | $O(\log n)$

- 셋에서 특정 값을 가진 데이터를 찾아 제거하며, 자동으로 오름차순 정렬을 유지함.
- 단순히 set.erase()을 수행시킬 수도 있고, 이를 출력할 수도 있음.
- 삭제 성공 시 1, 실패 시 0을 반환함.

set.size() | <set> | $O(1)$

- 셋에 현재 저장되어 있는 데이터의 총 개수를 확인하는 함수
- 내부 컨테이너에 요소의 수를 저장하고 있어서 $O(1)$ 의 시간 복잡도를 가짐

2. Set

```

1  #include <iostream>
2  #include <set>
3
4  using namespace std;
5
6  int main() {
7      set<int> s = {5, 2, 3, 2, 6};
8
9      if(s.find(2) != s.end()) s.erase(2);
10     // {3, 5, 6}
11
12     cout << s.count(2) << "\n";
13     cout << s.count(3) << "\n";
14
15     auto it = s.lower_bound(5);
16
17     if(it != s.end()) cout << "Found\n";
18     else cout << "Not Found\n";
19
20     it = s.upper_bound(6);
21
22     if(it != s.end()) cout << *it;
23     else cout << "Not Found";
24
25     return 0;
26 }
```

set.find(val) | <set> | $O(\log n)$

- 셋에서 특정 값을 가진 데이터를 찾는 함수
- 데이터를 찾았을 경우 해당 요소의 반복자를 반환하며, 찾지 못했을 경우 set.end()를 반환

set.count(val) | <set> | $O(\log n)$

- 셋에서 특정 값을 가진 데이터의 개수를 확인하는 함수
- 셋은 중복 원소를 허용하지 않으므로 데이터가 존재하면 1, 없으면 0을 반환

set.lower_bound(val), set.upper_bound(val) | <set> | $O(\log n)$

- 셋의 원소들 중 인자로 전달된 값 이상의/초과하는 첫 번째 원소의 반복자를 반환함.
- 데이터를 찾지 못했을 경우 set.end()를 반환
- unordered_set에서는 사용 불가능

set.clear() | $O(n)$

- 셋의 내부를 초기화하는 함수

```

Input:                                     Copy
-----
Expected Output:                           Copy
0
1
Found
6
```

2. Set

```
1  #include <iostream>
2  #include <set>
3
4  using namespace std;
5
6  int main() {
7      set<int> s = {40, 10, 30, 20, 50};
8
9      s.insert(25);
10
11     for (const int& n : s) {
12         cout << n << " ";
13     }
14
15     return 0;
16 }
```

범위 기반 for문

- vector, string과 마찬가지로 set에도 범위 기반 for문을 사용하여 요소를 순회할 수 있음

const

- 데이터를 읽기 전용으로 설정하여 프로그램 실행 중 값이 수정되는 것을 방지
 - 원소가 수정되면 정렬 순서가 망가지기 때문에, 애초에 원소를 읽기 전용으로 관리

&(참조자)

- 기존 변수에 새로운 이름을 붙여 메모리 주소를 직접 공유하는 방식
- 데이터를 복사하지 않고 원본에 접근하므로 메모리 사용량을 줄이고 성능을 향상시킴
 - 원소가 수정되면 정렬 순서가 망가지기 때문에, 일반 참조자 사용을 막아 둠
 - 참조자를 사용하려면 명시적으로 const를 함께 써주어야 함

Input: [Copy](#)

Expected Output: [Copy](#)

10 20 25 30 40 50

2. Set: 실습(BOJ)



실습

- [10815 숫자 카드](#)
 - [코드](#)
- [7785 회사에 있는 사람](#)
 - [코드](#)

2. Set: 실습(정올)



실습

- [4637 Tutorial: STL Set 1](#)
 - 코드
- [4638 Tutorial: STL Set 2](#)
 - 코드
- [4639 Tutorial: STL Set 3](#)
 - 코드
- [4640 Tutorial: STL Set 4](#)
 - 코드

2. Map



Map(맵)

- 키(Key)와 값(Value)이 한 쌍을 이루어 데이터를 저장하는 자료구조
- 중복된 키를 허용하지 않음.
- 내부적으로는 레드 블랙 트리(Red-Black Tree)로 구성되어 삽입과 동시에 키(Key)를 기준으로 자동 오름차순 정렬됨.

2. Map

```

1  #include <iostream>
2  #include <map>
3  #include <string>
4
5  using namespace std;
6
7  int main() {
8      map<int, string> m1 = {
9          {1, "Banana"},
10         {5, "Apple"},
11         {121, "WaterMelon"}
12     };
13
14     m1.insert({7, "Melon"});
15     m1[90] = "Pear";
16
17     cout << m1[7] << "\n";
18     cout << m1[90] << "\n";
19
20     map<int, int> m2 = {
21         {1, 10},
22         {3, 5}
23     };
24
25     m2[2]++;
26     m2[2] += 2;
27
28     cout << m2[2] << "\n";
29
30     return 0;
31 }

```

map<key 자료형, value 자료형> 변수명 | <map>

- 맵을 선언하기 위해서는 <map> 라이브러리 필요
- 자료형에는 거의 모든 자료형을 사용할 수도 있으며, 구조체를 선언하여 자료형으로 넣을 수도 있음

m.insert({key, value}) | <map> | $O(\log n)$

- 맵에 새로운 데이터를 추가하여 저장하며, key를 기준으로 자동으로 오름차순 정렬을 유지함
- 맵에 이미 존재하는 key를 삽입할 경우 무시됨

m.erase(key) | <map> | $O(\log n)$

- 맵에서 특정 key를 가진 데이터를 찾아 제거하며, 자동으로 오름차순 정렬을 유지함.
- 단순히 m.erase()을 수행시킬 수도 있고, 이를 출력할 수도 있음.
- 삭제 성공 시 1, 실패 시 0을 반환함.

m.size() | <map> | $O(1)$

- 맵에 현재 저장되어 있는 데이터의 총 개수를 확인하는 함수
- 내부 컨테이너에 요소의 수를 저장하고 있어서 $O(1)$ 의 시간 복잡도를 가짐

```

Input:                                     Copy
-----
Expected Output:                           Copy
Melon
Pear
3

```

2. Map

```

1  #include <iostream>
2  #include <map>
3  #include <string>
4
5  using namespace std;
6
7  int main() {
8      map<int, string> m = {
9          {1, "Banana"},
10         {5, "Apple"},
11         {121, "WaterMelon"},
12         {84, "Pear"},
13         {74, "Mango"}
14     };
15
16     if(m.find(5) != m.end()) cout << m[5] << "\n";
17     else cout << "Can't find\n";
18
19     if(m.find(3) != m.end()) cout << m[3] << "\n";
20     else cout << "Can't find\n";
21
22     if(m.count(121)) cout << m[121] << "\n";
23
24     // key값이 50이상 100이하인 모든 key, value 출력
25     auto it1 = m.lower_bound(5);
26     auto it2 = m.upper_bound(100);
27
28     for(auto it = it1; it != it2; ++it) {
29         cout << (*it).first << " : " << (*it).second << "\n";
30     }
31
32     return 0;
33 }

```

`m.find(key)` | `<map>` | $O(\log n)$

- 맵에서 특정 key를 가진 데이터를 찾는 함수
- 데이터를 찾았을 경우 해당 요소의 반복자를 반환하며, 찾지 못했을 경우 `set.end()`를 반환

`m.count(key)` | `<map>` | $O(\log n)$

- 맵에서 특정 key를 가진 데이터의 개수를 확인하는 함수
- 맵은 중복 key를 허용하지 않으므로 데이터가 존재하면 1, 없으면 0을 반환

`m.lower_bound(key)`, `m.upper_bound(key)` | `<map>` | $O(\log n)$

- 맵의 원소들 중 인자로 전달된 값 이상의/초과하는 첫 번째 key의 반복자를 반환함.
- 데이터를 찾지 못했을 경우 `m.end()`를 반환
- `unordered_map`에서는 사용 불가능

`map.clear()` | $O(n)$

- 맵의 내부를 초기화하는 함수

```

Input:                                     Copy
-----
Expected Output:                           Copy
Apple
Can't find
WaterMelon
5 : Apple
74 : Mango
84 : Pear

```

2. Map

```

1  #include <iostream>
2  #include <map>
3  #include <string>
4
5  using namespace std;
6
7  int main() {
8      map<int, string> m = {
9          {1, "Banana"},
10         {84, "Pear"},
11         {74, "Mango"}
12     };
13
14     // pair 컨테이너 이용
15     for(const auto& kv : m) {
16         cout << kv.first << " : " << kv.second << "\n";
17     }
18
19     // 구조 분해 할당 이용
20     for(const auto& [key, value] : m) {
21         cout << key << " : " << value << "\n";
22     }
23
24     return 0;
25 }

```

Input: [Copy](#)

Expected Output: [Copy](#)

```

1 : Banana
74 : Mango
84 : Pear
1 : Banana
74 : Mango
84 : Pear

```

범위 기반 for문

- vector, string과 마찬가지로 map에도 범위 기반 for문을 사용하여 요소를 순회할 수 있음

const

- 데이터를 읽기 전용으로 설정하여 프로그램 실행 중 값이 수정되는 것을 방지
 - 원소가 수정되면 정렬 순서가 망가지기 때문에, 애초에 원소를 읽기 전용으로 관리

&(참조자)

- 기존 변수에 새로운 이름을 붙여 메모리 주소를 직접 공유하는 방식
- 데이터를 복사하지 않고 원본에 접근하므로 메모리 사용량을 줄이고 성능을 향상시킴
 - 원소가 수정되면 정렬 순서가 망가지기 때문에, 일반 참조자 사용을 막아 둠
 - 참조자를 사용하려면 명시적으로 const를 함께 써주어야 함

2. Map

일대 다 매핑

- 하나의 key에 여러 개의 value가 매핑되는 구조

map<key, vector<value>>

- value가 vector 안에 저장되는 구조이기 때문에 key를 통한 Lookup*이후 벡터에 접근하여 인덱싱할 수 있음

```
1  #include <iostream>
2  #include <map>
3  #include <vector>
4  #include <string>
5
6  using namespace std;
7
8  int main() {
9      map<int, vector<string>> m = {
10         {0, {"one", "two", "three"}},
11         {1, {"spring", "summer"}}
12     };
13
14     m[0].push_back("four");
15     m[1].push_back("autumn");
16
17     for(auto it : m) {
18         cout << it.first << " : ";
19
20         for(int i = 0; i < it.second.size(); i++) {
21             cout << it.second[i] << " ";
22         }
23
24         cout << "\n";
25     }
26
27     return 0;
28 }
```

Input: [Copy](#)

Expected Output: [Copy](#)

```
0 : one two three four
1 : spring summer autumn
```

* Lookup(조회, 검색) : Key를 통해 Value를 찾아내는 행위

2. Map: 실습(BOJ)



실습

- [10816 숫자 카드 2](#)
 - [코드](#)
- [17219 비밀번호 찾기](#)
 - [코드](#)

2. Map: 실습(정올)



실습

- [8561 Tutorial : STL Map 1](#)
 - 코드
- [8062 Tutorial : STL Map 2](#)
 - 코드
- [8036 평행하는 선분의 수](#)
 - 코드
- [8039 실시간 투표현황](#)
 - 코드

3. Priority Data Structures

3. Priority Data Structures



Priority Data Structures(우선순위 자료구조)

- 물리적인 선형 순서가 아니라, 데이터가 가진 고유한 우선순위를 기준으로 정렬되고 제어되는 자료구조
- 대기열이나 그래프 알고리즘 등에서 매 순간 가장 먼저 처리해야 할 최적의 데이터(최댓값/최솟값)를 빠르게 추출
- priority queue, heap이 이에 해당 됨

3. Priority Queue

Priority Queue(우선순위 큐)

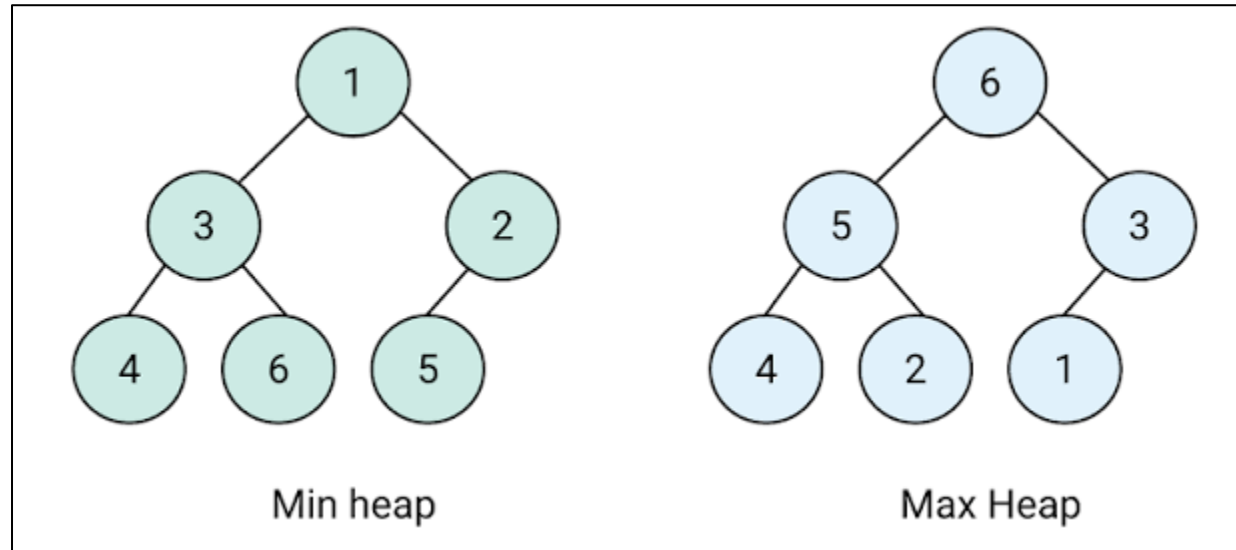
- 먼저 들어간 데이터가 먼저 나오는 일반적인 큐와 달리, 데이터마다 부여된 중요도가 가장 높은 것부터 먼저 나오는 자료구조
- '가장 높은 중요도를 가진 데이터를 먼저 꺼낸다'는 방식 그 자체를 뜻하므로, 이 규칙만 만족한다면 내부를 어떤 방식으로 프로그래밍하든 상관없음
- 즉, 배열, 리스트, 힙으로 구현할 수 있음.

표현 방법	삽입	삭제
순서 없는 배열	$O(1)$	$O(n)$
정렬된 배열	$O(1)$	$O(n)$
순서 없는 연결 리스트	$O(n)$	$O(1)$
정렬된 연결 리스트	$O(n)$	$O(1)$
힙(heap)	$O(\log n)$	$O(\log n)$

3. Heap

Heap(힙)

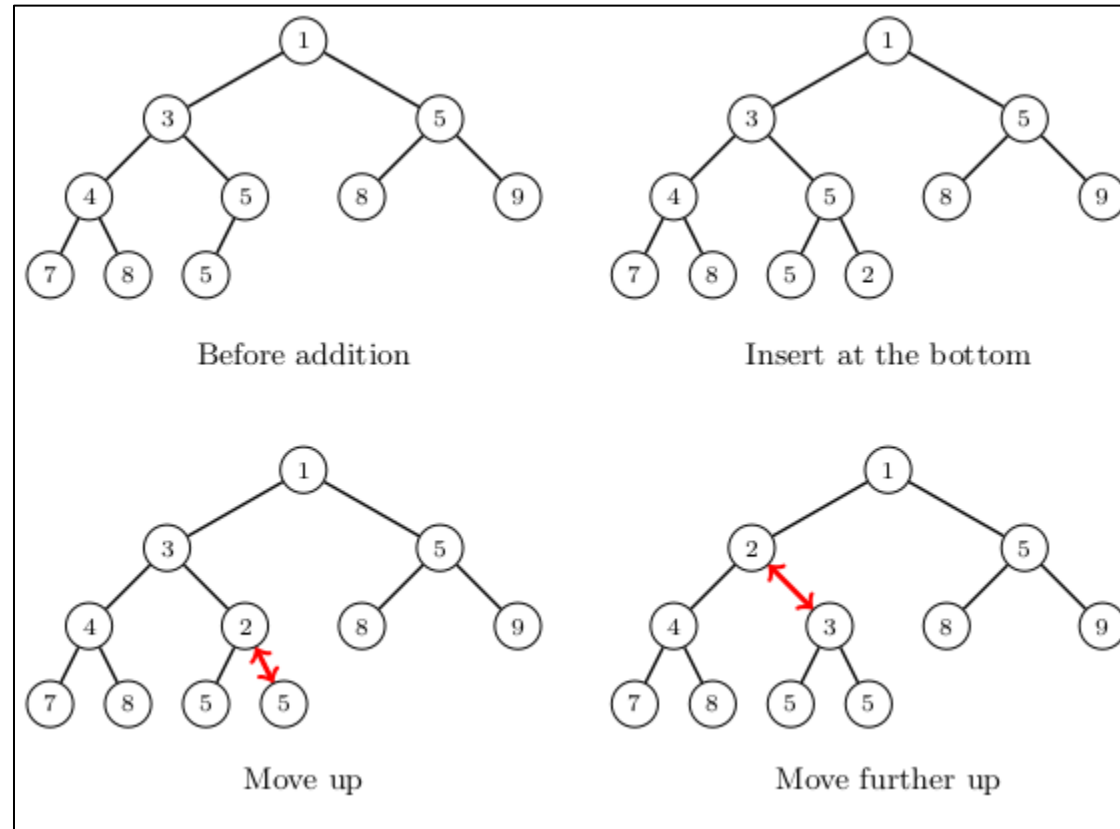
- 완전 이진 트리의 일종
 - 마지막 레벨을 제외한 모든 레벨이 노드로 꽉 차있음.
 - 마지막 레벨의 노드들은 왼쪽부터 차례대로 채워짐.
- 여러 개의 값들 중에서 최댓값이나 최솟값을 빠르게 찾아내도록 만들어진 자료구조
- 부모 데이터가 자식 데이터보다 항상 크거나(최대 힙), 항상 작은(최소 힙) 일관된 대소관계를 유지함



3. Heap

Heap(힙)

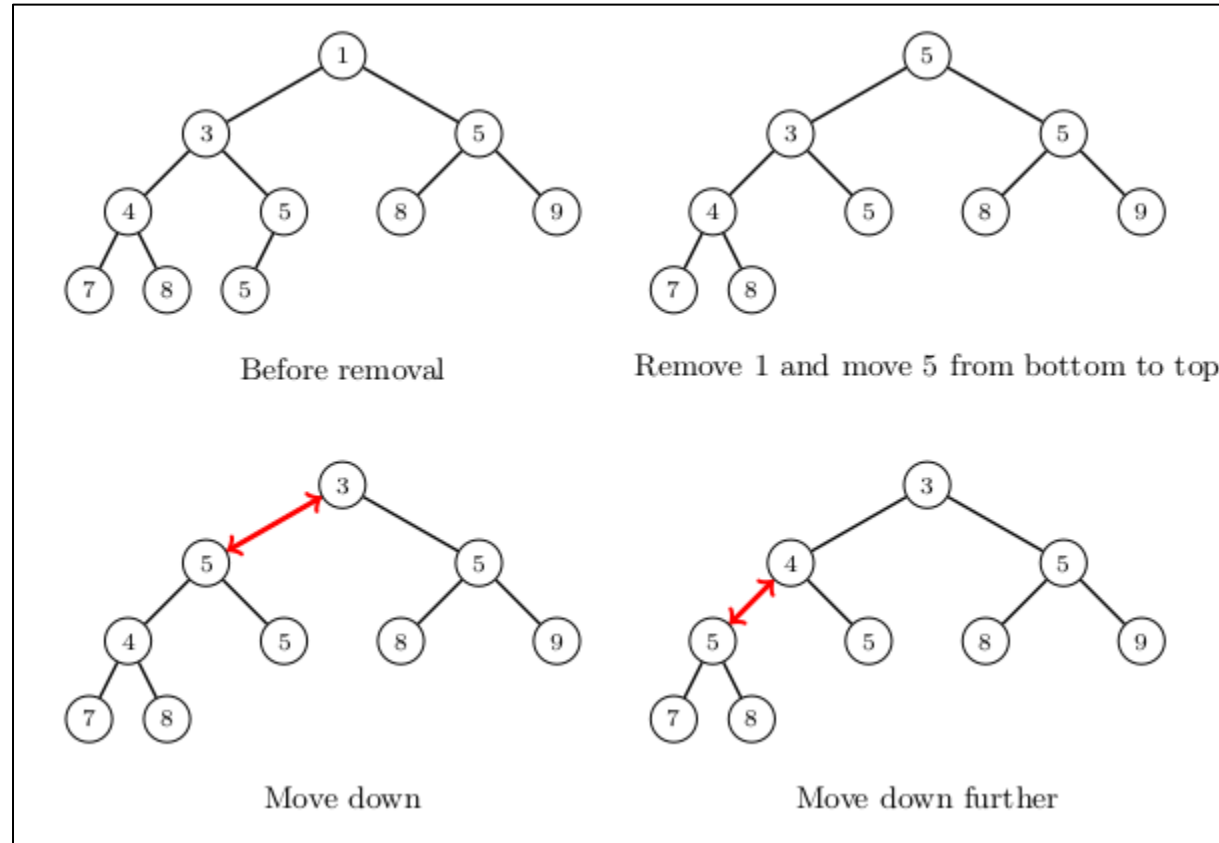
- 데이터의 삽입 : $O(\log n)$



3. Heap

Heap(힙)

- 데이터의 삭제 : $O(\log n)$



3. Priority Queue

```

1  #include <iostream>
2  #include <queue>
3
4  using namespace std;
5
6  int main() {
7      priority_queue<int> pq;
8
9      int n, num;
10
11     cin >> n;
12
13     for(int i = 0; i < n; ++i) {
14         cin >> num;
15
16         pq.push(num);
17     }
18
19     cout << pq.size() << "\n";
20
21     while(!pq.empty()) {
22         cout << pq.top() << " ";
23
24         pq.pop();
25     }
26
27     return 0;
28 }

```

priority_queue<자료형> 변수명 | <queue>

- 우선순위 큐를 선언하기 위해서는 <queue> 라이브러리 필요
- 자료형에는 거의 모든 자료형을 사용할 수도 있으며, 구조체를 선언하여 자료형으로 넣을 수도 있음

pq.push(value) | <queue> | $O(\log n)$

- 새로운 데이터를 추가하고, 설정된 우선순위를 유지
- 데이터가 추가될 때마다 최적의 자리를 찾아 트리의 높이만큼 부모 노드와 위치를 바꾸는 연산 발생

pq.pop() | <queue> | $O(\log n)$

- 현재 가장 우선순위가 높은(최상단 루트 노드에 있는) 데이터를 제거
- 데이터를 삭제한 후, 힙 구조를 유지하기 위해 정렬하는 연산이 발생
- 비어 있는 우선순위 큐에 pop를 사용하면 프로그램이 정지되며 아무것도 출력되지 않음

입력:	복사
6	
10 7 99 1 43 7	

예상 출력:	복사
6	
99 43 10 7 7 1	

3. Priority Queue

```

1  #include <iostream>
2  #include <queue>
3
4  using namespace std;
5
6  int main() {
7      priority_queue<int> pq;
8
9      int n, num;
10
11     cin >> n;
12
13     for(int i = 0; i < n; ++i) {
14         cin >> num;
15
16         pq.push(num);
17     }
18
19     cout << pq.size() << "\n";
20
21     while(!pq.empty()) {
22         cout << pq.top() << " ";
23
24         pq.pop();
25     }
26
27     return 0;
28 }

```

`pq.top()` | `<queue>` | $O(1)$

- 현재 가장 우선순위가 높은(최상단 루트 노드에 있는) 데이터를 확인하는 함수
- 비어 있는 우선순위 큐에 top를 사용하면 프로그램이 정지되며 아무것도 출력되지 않음

`pq.size()` | `<queue>` | $O(1)$

- 현재 저장되어 있는 데이터의 총 개수를 확인하는 함수
- 내부 컨테이너에 요소의 수를 저장하고 있어서 $O(1)$ 의 시간 복잡도를 가짐

`pq.empty()` | `<queue>` | $O(1)$

- 우선순위 큐에 저장된 데이터가 없는지 여부를 확인하는 함수
- bool형으로 반환하며, 큐가 비어 있으면 true, 요소가 하나라도 존재한다면 false를 반환

우선순위 큐의 초기화 | $O(n)$

- 우선순위 큐에는 `pq.clear()` 함수가 존재하지 않음
- 우선순위 큐를 초기화하기 위해서는 재선언 하거나 `pq = {}`으로 초기화할 수 있음
- 단, `pq = {1, 2, 3}` 같은 방법은 통하지 않고, `pq = {}`만 예외적으로 가능함.

입력:	복사
6	
10 7 99 1 43 7	
예상 출력:	복사
6	
99 43 10 7 7 1	

3. Priority Queue

priority_queue<자료형, vector<자료형>, greater<>> 변수명

- 최소힙을 만들기 위해서는 다음과 같은 선언 필요
- 그 외, 구조체나 pair 자료구조를 사용한 정렬 방법은 정렬 파트에서 다룰 예정

```
1  #include <iostream>
2  #include <queue>
3
4  using namespace std;
5
6  int main() {
7      priority_queue<int, vector<int>, greater<>> pq;
8
9      int n, num;
10
11     cin >> n;
12
13     for(int i = 0; i < n; ++i) {
14         cin >> num;
15
16         pq.push(num);
17     }
18
19     cout << pq.size() << "\n";
20
21     while(!pq.empty()) {
22         cout << pq.top() << " ";
23
24         pq.pop();
25     }
26
27     return 0;
28 }
```

입력:	복사
6	
10 7 99 1 43 7	
예상 출력:	복사
6	
1 7 7 10 43 99	

3. Priority Queue: 실습(정올)



실습

- [1089 힙정렬](#)
 - 코드
- [2082 힙정렬2](#)
 - 코드
- [8589 하늘에서 떨어지는 돈](#)
 - 코드
- [4905 STL - Priority Queue](#)
 - 코드